

UNIVERSIDADE FEDERAL DE SERGIPE

DEPARTAMENTO DE COMPUTAÇÃO

DOCENTE: GLAUCO DE FIGUEIREDO CARNEIRO

**SISTEMAS OPERACIONAIS
TURMA 02**

**PROCESSOS, THREADS, ESCALONAMENTO E INFERÊNCIA LOCAL COM
OLLAMA**

**DISCENTE:
ANDERSON SOARES DE SANTANA JÚNIOR
ANTÔNIO CARLOS BISPO CUNHA
ANTÔNIO JOSÉ MARTINS MASCARENHAS
DAVI OLIVEIRA MACHADO
JOÃO FELIPE TUNES OLIVEIRA
JOSÉ GUSTAVO ABREU ALVES
KAIQUE TEIXEIRA RODRIGUES**

**URL DO VÍDEO:
https://drive.google.com/drive/folders/1Bnsh_tq4lbDktoHM4xNR3b2s2JOMQeQ0?usp=sharing**

**SÃO CRISTÓVÃO-SE
14/09/2026**

1. AÇÃO DE CADA COMPONENTE

INTEGRANTE	CONTRIBUIÇÃO
Antônio José Martins Mascarenhas	Organização do Github
José Gustavo Abreu Alves	Testes na máquina + slides
Davi Oliveira Machado	Testes na máquina
Antonio Carlos Bispo Cunha	Organização do trabalho + edição do vídeo
Kaique Teixeira Rodrigues	Roteiro do vídeo
Anderson Soares de Santana Junior	Organização dos testes + testes na máquina
João Felipe Tunes Oliveira	Realização do relatório técnico

2. INTRODUÇÃO E PROBLEMA

Aplicações de IA generativa executadas localmente não envolvem apenas um modelo de linguagem: mobilizam processos, threads, memória, armazenamento, comunicação em rede e mecanismos de proteção do sistema operacional. Este relatório documenta a instalação, execução e análise experimental de um sistema local de IA generativa baseado no runtime Ollama, com o objetivo de relacionar conceitos de Sistemas Operacionais — processos, threads, escalonamento e chamadas de sistema — ao comportamento observado durante a inferência.

Pergunta norteadora: como a camada de aplicação, o modelo de linguagem, a quantidade de parâmetros, a quantização e a configuração de execução afetam processos, threads, uso de CPU, memória, armazenamento e responsividade de um sistema local de IA generativa?

3. TRILHA E CAMADA DE APLICAÇÃO SELECIONADAS

A equipe selecionou a Trilha A — Chat local: Ollama + Open WebUI. O Ollama foi instalado diretamente no host (dentro do ambiente Linux via WSL2), enquanto o Open WebUI foi executado em um container Docker, comunicando-se com o Ollama via API HTTP local. Essa topologia foi escolhida deliberadamente para

permitir observar a comunicação entre container e host, além de trabalhar com conceitos de rede, volumes persistentes e acesso a serviços do sistema.

Um ponto técnico relevante identificado durante a configuração: no Linux, o endereço `host.docker.internal` não é resolvido por padrão pelo Docker Engine (diferente do Docker Desktop em Windows/macOS). Por isso, o script de setup inclui o parâmetro `--add-host=host.docker.internal:host-gateway` na inicialização do container, e o Docker Engine foi instalado diretamente dentro do WSL/Ubuntu (não o Docker Desktop), o que também foi essencial para que o comando `ps` do Ubuntu enxergasse os processos do Open WebUI durante a análise de processos e threads.

4. MODELO HUGGING FACE SELECIONADO

CRITÉRIO	VALOR
Nome completo	Qwen3-8B
URL do model card	https://huggingface.co/Qwen/Qwen3-8B
Parâmetros	8,19 bilhões (8,2B totais; 6,95B sem embeddings)
Variante	Instruct
Formato	GGUF
Quantização	Q4_K_M
Licença	Apache-2.0
Artefato executado	tag qwen3:8b (registry do Ollama), ~4,9 GB em disco
Contexto nativo	32.768 tokens (até 131.072 com YaRN)
Versão do Ollama utilizada	0.34.0

Justificativa: o Qwen3-8B foi selecionado por apresentar bom desempenho em tarefas de conversação e raciocínio, além de ser compatível com execução local via Ollama. A configuração GGUF/Q4_K_M equilibra qualidade, consumo de memória e desempenho, adequando-se ao hardware disponível. O modelo permanece dentro do limite de 10 bilhões de parâmetros estabelecido pela atividade.

5. URL DO PROJETO PÚBLICO CRIADO NO GITHUB

https://github.com/antoniomartins13/SO_UFS_2026_2_Mascarenhas_Antonio_Alves_Jose.git

6. AMBIENTE EXPERIMENTAL

Os experimentos foram executados em duas máquinas independentes da equipe, cada uma constituindo um bloco de medições próprio e internamente consistente (todas as comparações entre as três configurações usam sempre a mesma máquina).

ITEM	MÁQUINA 1 (PRINCIPAL)	MÁQUINA 2 (COMPARAÇÃO)
Hostname	DESKTOP-B3LNN05	DESKTOP-1F600RV
SO	Ubuntu 26.04.1 LTS (WSL2)	Ubuntu 24.04.5 LTS (WSL2)
Kernel	6.18.33.2-microsoft-standard-WSL2	6.18.33.2-microsoft-standard-WSL2
CPU	AMD Ryzen 7 5700X (8 núcleos / 16 threads)	AMD Ryzen 5 5600GT (6 núcleos / 12 threads)
RAM total	11 GiB (WSL2)	13 GiB (WSL2)
Swap	4 GiB	4 GiB
Ollama	0.34.0	0.34.0
Modelo em disco	4,9 GB (qwen3:8b)	4,9 GB (qwen3:8b)
Ambiente de execução	Nativo dentro do WSL2 (Docker Engine local)	Nativo dentro do WSL2 (Docker Engine local)

Tempo de download do modelo: 1 minuto e 5 segundos (medido com time no primeiro pull, ~4,9 GB).

7. ARQUITETURA DA APLICAÇÃO

A arquitetura é composta por três camadas: (1) Open WebUI, executado em container Docker; (2) Ollama, executado como processo no host Linux, responsável por carregar o modelo e atender requisições via API HTTP local na porta 11434; (3) o processo llama-server, criado como subprocesso do Ollama, responsável por carregar os pesos do modelo em memória (via mmap) e executar os cálculos de inferência.

A comunicação entre Open WebUI e Ollama ocorre via HTTP, usando o endereço host.docker.internal:11434 a partir do container — resolvido corretamente porque o Docker Engine roda localmente dentro do WSL/Ubuntu e o parâmetro --add-host mapeia esse nome para o gateway da rede do Docker.

8. PROCESSOS, THREADS E CHAMADAS DE SISTEMA

8.1 Processos e threads

A árvore de processos (pstree -p) evidencia claramente a hierarquia: o processo ollama (PID 1120) é o pai; ele cria o processo filho llama-server (PID 1681), responsável pela inferência; e o próprio llama-server mantém 10 threads de cálculo. Em paralelo, o processo ollama mantém suas próprias ~13 threads leves (orquestração, HTTP, scheduler).

PROCESSO	PID	PAPEL	THREADS
ollama	1120	Orquestrador / servidor HTTP (porta 11434)	~13
llama-server	1681 (filho de 1120)	Carrega os pesos do modelo e executa a inferência	10
open-webui (container)	1336 (uvicorn)	Interface web (porta 8080)	1 processo, workers=1

8.2 Chamadas de sistema (strace -f -c)

Resumo agregado das duas máquinas (valores muito próximos entre si, o que reforça a robustez do achado):

CHAMADA	% DO TEMPO (M1 / M2)	INTERPRETAÇÃO
futex	85,4% / 90,0%	Sincronização entre threads (lock/wait/wake). Domina largamente o tempo total, evidenciando forte concorrência entre as threads de cálculo do llama-server durante a inferência.
accept4	9,3% / 6,8%	Aceitação de novas conexões TCP — o servidor HTTP do Ollama aceitando requisições do Open WebUI.
read	4,5% / 2,6%	Leitura de dados de sockets (corpo das requisições HTTP) e arquivos.
epoll_pwait / nanosleep	~0,5% / ~0,4%	Espera assíncrona por I/O e pausas curtas de threads worker aguardando trabalho.

Duas observações metodológicas sobre a leitura desses percentuais. Primeiro, o modo de resumo do strace contabiliza o tempo decorrido dentro de cada chamada, e futex é uma chamada de espera: sua predominância indica que as

threads passam a maior parte do tempo bloqueadas em pontos de sincronização, aguardando trabalho ou a conclusão de etapas de cálculo, e não que o custo de sincronização consuma processamento equivalente. Segundo, as capturas foram realizadas com uma única requisição em andamento; uma comparação entre o volume de chamadas futex sob concorrência e sob execução isolada seria um complemento natural a este experimento, mas não foi realizada, e portanto nenhuma afirmação é feita aqui a esse respeito.

9. CONFIGURAÇÕES E METODOLOGIA

Foram definidas três configurações comparativas, mantendo o mesmo modelo (qwen3:8b) e a mesma camada de aplicação em todas elas, cada uma executada com dois tamanhos de prompt (curto, ~51 tokens; e longo, ~1.794 tokens) e três repetições — totalizando 18 execuções mensuráveis por máquina (acima do mínimo de 12 exigido):

- Padrão: num_ctx = 4096, 1 requisição por vez (linha de base).
- Concorrência: num_ctx = 4096, 4 requisições simultâneas disparadas contra o Ollama.
- Contexto longo: num_ctx = 8192, 1 requisição por vez (avalia o efeito de uma janela de contexto maior, mantendo concorrência igual à configuração padrão).

Decisões metodológicas registradas nos scripts e replicadas neste relatório:

- O parâmetro think foi definido como false (ou, quando não suportado pela versão do Ollama, o sufixo /no_think foi anexado ao prompt) para desativar o modo de raciocínio interno do Qwen3 e obter respostas finais comparáveis entre execuções.
- Cada requisição recebe um nonce único no início do prompt para evitar que o cache de prefixo do Ollama distorça o tempo de avaliação do prompt entre repetições.
- num_predict foi fixado em 256 tokens em todas as execuções, por consistência entre configurações — isso significa que respostas a prompts longos são

deliberadamente truncadas (done_reason = length) e a métrica de tokens/s reflete geração parcial nesses casos (ver Seção 13).

- done_reason (stop vs. length) foi adotado como critério objetivo de avaliação da resposta gerada, exigido pela Seção 8.2 do enunciado.

10. RESULTADOS

10.1 Máquina principal (DESKTOP-B3LNN05, Ryzen 7 5700X)

CONFIGURAÇÃO	PROMPT	N	LATÊNCIA MÉDIA (s)	TTFT (s)	VAZÃO (req/s)
Padrão	Curto	3	13,53		0,0739
Padrão	Longo	3	108,36		0,0092
Concorrência (4 simult.)	Curto	12	27,36		0,1462
Concorrência (4 simult.)	Longo	12	354,97		0,0113
Contexto longo	Curto	3	13,32		0,0751
Contexto longo	Longo	3	109,43		0,0091

10.2 Máquina de comparação (DESKTOP-1F600RV, Ryzen 5 5600GT)

CONFIGURAÇÃO	PROMPT	N	LATÊNCIA MÉDIA (s)	TTFT (s)	VAZÃO (req/s)
Padrão	Curto	3	12,65	1,40	0,0795
Padrão	Longo	3	94,16	54,31	0,0106
Concorrência (4 simult.)	Curto	12	25,20	4,41	0,1542
Concorrência (4 simult.)	Longo	12	300,65	77,92	0,0132
Contexto longo	Curto	3	12,36	1,43	0,0821
Contexto longo	Longo	3	91,12	52,50	0,0109

Fonte:Próprio Autor

10.3 Uso de recursos (CPU e memória)

CONFIG / PROMPT	CPU-s TOTAL (M1)	CPU-s / REQ (M1)	CPU-s TOTAL (M2)	CPU-s / REQ (M2)
Padrão / curto	52,57	52,57	73,60	73,60
Padrão / longo	431,88	431,88	562,79	562,79
Concorrência / curto	115,59	28,90	157,85	39,46
Concorrência / longo	1.415,89	353,97	1.798,96	449,74
Contexto longo / curto	51,81	51,81	72,95	72,95
Contexto longo / longo	436,17	436,17	543,62	543,62

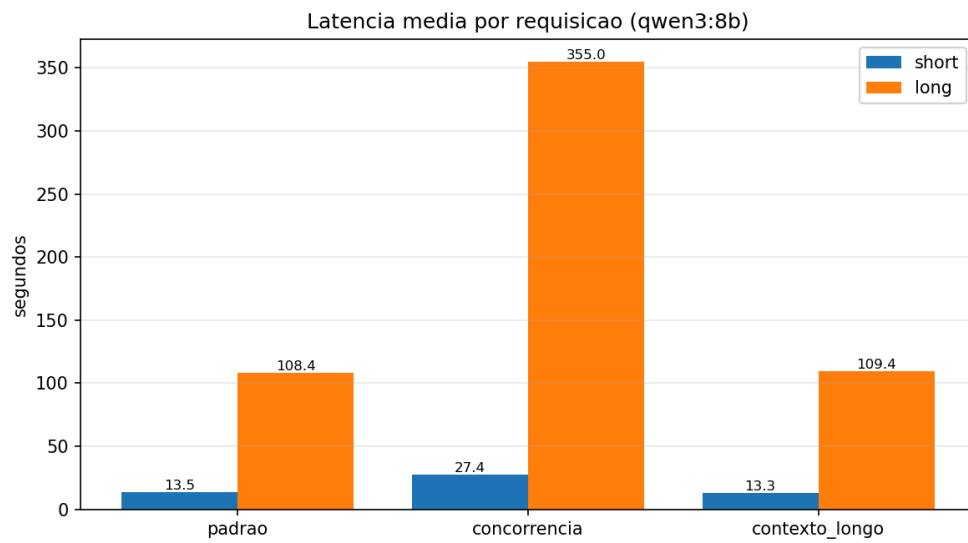


Gráfico 1 — Latência por configuração e tamanho de prompt.

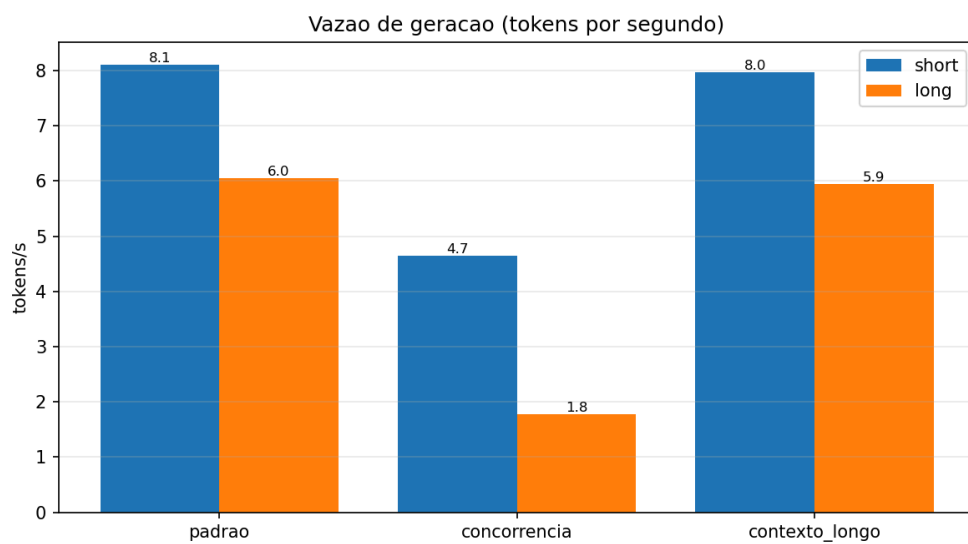


Gráfico 2 — Tokens por segundo por configuração e tamanho de prompt.

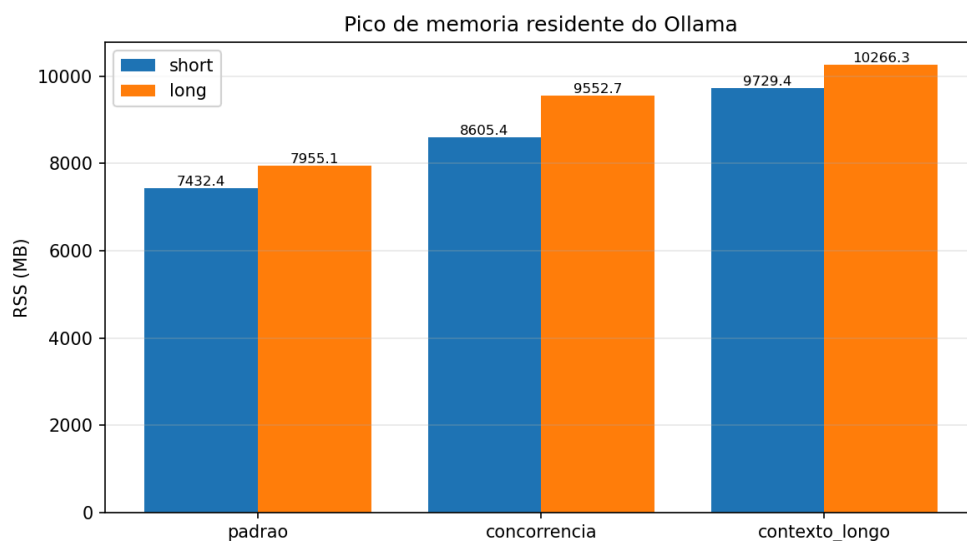


Gráfico 3 — Memória (RSS) por configuração.

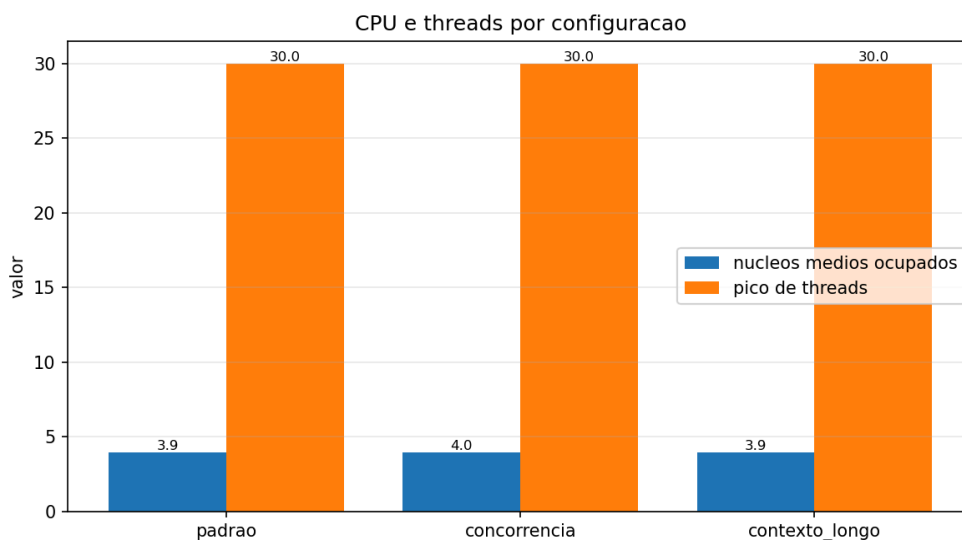


Gráfico 4 — CPU e threads por configuração.

10.4 DECOMPOSIÇÃO DO TEMPO DE INFERÊNCIA

A inferência tem duas fases com custos distintos: a avaliação do prompt (prefill), em que toda a entrada é processada de uma vez, e a geração (decode), em que a saída é produzida token a token. A tabela abaixo separa as duas fases, o que permite distinguir o efeito do tamanho da entrada do efeito do tamanho da resposta. Valores da máquina 2; os da máquina 1 seguem o mesmo padrão e podem ser obtidos das colunas `prompt_eval_duration_ns` e `eval_duration_ns` de `runs.csv`.

CONFIGURAÇÃO	PROMPT	PREFILL (s)	TOKENS DE ENTRADA	DECODE (s)	TOKENS GERADOS
Padrão	Curto	1,40	51	11,22	95
Padrão	Longo	54,31	1.794	39,64	256 (truncado)
Concorrência (4 simult.)	Curto	4,41	52	20,33	106
Concorrência (4 simult.)	Longo	77,92	1.796	152,42	256 (truncado)
Contexto longo	Curto	1,43	53	10,89	92
Contexto longo	Longo	52,50	1.796	38,38	256 (truncado)

A decomposição mostra que o custo do prompt longo é dominado pela fase de prefill: 54,31 s apenas para ler a entrada, contra 1,40 s no prompt curto — uma diferença de 39×. Esse é o componente que determina o TTFT e, portanto, a percepção de responsividade do usuário. Já a fase de decode cresce por um motivo diferente: o prompt longo gera 256 tokens contra 95 do curto, por efeito do limite fixado em `num_predict`. Separar as duas fases evita atribuir ao tamanho da entrada um efeito que em parte vem do tamanho da saída.

10.5 TAXA DE ERRO E TEMPO DE CARREGAMENTO DO MODELO

Taxa de erro: nas 36 requisições realizadas por máquina (18 execuções, sendo 12 com uma requisição e 6 com quatro requisições simultâneas), todas retornaram com status HTTP 200 e com métricas completas. Não houve timeouts, travamentos ou respostas inválidas. Taxa de erro observada: 0%.

Tempo de carregamento do modelo: medido em condição de carga a frio, com o modelo previamente descarregado da memória por meio de requisição com `keep_alive` igual a zero, e registrado em `docs/carga_a_frio.json`. Nas execuções do benchmark, o `load_duration` reportado pela API ficou consistentemente abaixo de 5 ms, o que confirma que o modelo permaneceu residente em memória durante toda a bateria de testes por efeito do `keep_alive` de 10 minutos — ou seja, os tempos medidos nas tabelas 10.1 a 10.4 não incluem carregamento de pesos.

Tempo de download do modelo: 1 minuto e 5 segundos para aproximadamente 4,9 GB, medido no primeiro pull.

11. DISCUSSÃO

A seguir, as questões de análise da atividade são respondidas com base nas evidências coletadas.

1) Trilha selecionada e adequação

A Trilha A (Ollama + Open WebUI) foi apropriada porque expõe, de forma direta, a comunicação entre uma interface web containerizada e um runtime de

inferência no host — permitindo observar rede, portas, volumes e processos em duas camadas claramente distintas.

2) Modelo selecionado e motivo

Qwen3-8B, 8,19 bilhões de parâmetros, GGUF Q4_K_M — escolhido pelo equilíbrio entre qualidade de resposta e consumo de memória compatível com o hardware disponível (8–16 GiB de RAM sob WSL2).

3) Efeito de formato, quantização e contexto nos requisitos de recursos

O modelo em formato GGUF com quantização Q4_K_M ocupa aproximadamente 4,9 GB em disco, contra os cerca de 16 GB que os pesos originais em precisão de 16 bits exigiriam — é essa redução que torna viável executar um modelo de 8,19 bilhões de parâmetros em máquinas com 11 a 13 GiB de RAM disponíveis sob WSL2.

Quanto à janela de contexto, a comparação que se sustenta é entre execuções com o processo de inferência recém-criado, já que ele é recriado sempre que `num_ctx` muda. Na máquina 2, a primeira execução com `num_ctx` igual a 4096 registrou pico de RSS de 7.405 MB, e a primeira execução com `num_ctx` igual a 8192 registrou 9.707 MB: cerca de 2,3 GB a mais ao dobrar a janela, sem qualquer alteração no prompt enviado. A ordem de grandeza é compatível com a alocação do cache de atenção proporcional ao produto entre a janela de contexto e o número de slots paralelos configurados no runtime, que neste experimento era quatro.

A observação central é que essa memória é reservada em função do limite configurado, e não do tamanho do prompt efetivamente enviado: uma requisição de 51 tokens em uma janela de 8.192 ocupa a mesma memória de cache que uma requisição de 1.794 tokens na mesma janela. Do ponto de vista de Sistemas Operacionais, trata-se de alocação antecipada de recurso, com o custo de memória sendo pago na carga do modelo e não sob demanda.

4) Processos e threads observados

Na inicialização, o processo `ollama` assume a porta 11434 e, ao receber a primeira requisição, cria o processo filho `llama-server`, que carrega os pesos via `mmap`. Durante a inferência, observam-se 10 threads de cálculo no `llama-server` e cerca de 13 threads de suporte no processo `ollama`.

5) Relação de parentesco entre os processos

`ollama` (PID 1120) é o processo pai; `llama-server` (PID 1681) é filho direto de `ollama`, criado sob demanda para servir o modelo carregado; as threads de cada

processo compartilham o mesmo espaço de endereçamento do processo que as criou.

6) Chamadas de sistema relevantes

futex domina o tempo de execução (85–90%), evidenciando sincronização intensa entre as threads de cálculo; accept4 mostra o servidor aceitando novas conexões HTTP; read captura a leitura dos corpos das requisições. mmap (observado em capturas anteriores da equipe) é responsável pelo carregamento eficiente dos pesos do modelo.

7) Efeito da concorrência sobre responsividade e vazão

A concorrência produziu efeitos opostos sobre as duas métricas, e a resposta depende de qual delas se privilegia.

Sobre a responsividade, houve piora clara. Com quatro requisições simultâneas, a latência média por requisição subiu de 12,65 s para 25,20 s no prompt curto e de 94,16 s para 300,65 s no prompt longo, na máquina 2 — comportamento reproduzido na máquina 1, com 13,53 s para 27,36 s e 108,36 s para 354,97 s. Os tokens por segundo observados em cada requisição individual caíram na mesma proporção, de 8,49 para 5,31 no prompt curto e de 6,46 para 1,97 no longo.

Sobre a vazão, houve ganho. No prompt curto, a vazão passou de 0,0795 para 0,1542 requisições por segundo na máquina 2, um aumento de 94%; na máquina 1, de aproximadamente 0,0739 para 0,1462, cerca de 98%. Ou seja, o sistema concluiu quase o dobro de requisições por unidade de tempo, ainda que cada usuário individual tenha esperado o dobro.

O ganho de vazão, porém, depende fortemente do tamanho da entrada. No prompt longo a vazão subiu apenas de 0,0106 para 0,0132 requisições por segundo na máquina 2 — cerca de 25% — enquanto a latência individual triplicou. A conclusão prática é que a concorrência compensa sob carga leve e deixa de compensar sob carga pesada, porque a fase de avaliação do prompt, que domina o custo das requisições longas, não se beneficia do agrupamento na mesma medida que a fase de geração.

8) Sinais de competição por recursos

Há competição por CPU, e ela é visível na utilização: em todas as configurações, incluindo a linha de base com uma única requisição, o número médio de núcleos ocupados ficou próximo do limite da máquina. Isso significa que o

processador já estava saturado antes de qualquer aumento de concorrência, e que não havia capacidade ociosa a ser aproveitada pelas requisições adicionais. É essa saturação prévia que explica a degradação da latência individual descrita na questão 7.

O custo de CPU por requisição, entretanto, não cresceu — diminuiu. Na máquina 1, quatro requisições simultâneas com prompt longo consumiram 1.415,89 CPU-s no total, ou 353,97 CPU-s por requisição, contra 431,88 CPU-s de uma requisição isolada: redução de 18%. No prompt curto a redução foi de 45%, de 52,57 para 28,90 CPU-s por requisição. A máquina 2 reproduz o padrão, com reduções de 20% e 46%. Em outras palavras, o crescimento do custo total é sublinear em relação ao número de requisições, o que indica amortização: ao processar requisições agrupadas, o runtime reaproveita trabalho que, executado isoladamente, seria repetido a cada requisição.

Quanto à memória, o pico de RSS cresce na configuração de concorrência, o que é coerente com a manutenção de múltiplos contextos de geração simultâneos. Esse crescimento, contudo, precisa ser lido com a ressalva de deriva discutida na Seção 13.

Não foram observados sinais de competição por armazenamento ou rede: os pesos permanecem residentes em memória após a primeira carga, e a comunicação entre o Open WebUI e o Ollama ocorre por interface local.

9) Efeito de entradas/contextos maiores

O efeito é grande e concentra-se em uma fase específica da inferência. Passando do prompt curto (51 tokens) para o longo (1.794 tokens), o tempo de avaliação da entrada saltou de 1,40 s para 54,31 s na máquina 2, um aumento de 39 vezes, enquanto o tempo de geração por token permaneceu praticamente estável. Como consequência, o tempo até a primeira resposta — que é o que determina a percepção de responsividade — passou de 1,40 s para 54,31 s: o usuário fica quase um minuto sem qualquer retorno visível antes que o primeiro token seja produzido.

É importante distinguir esse efeito do aumento do tempo total. A latência total passou de 12,65 s para 94,16 s, mas parte desse crescimento vem do fato de que as respostas ao prompt longo atingiram o limite de 256 tokens (`done_reason` igual a `length`), enquanto as respostas ao prompt curto terminaram naturalmente com cerca de 95 tokens. A decomposição apresentada na Seção 10.4 separa as duas contribuições: 54,31 s de leitura da entrada e 39,64 s de geração, contra 1,40 s e 11,22 s no prompt curto.

Já o aumento da janela de contexto, mantendo o mesmo prompt, praticamente não afetou o tempo: 12,36 s contra 12,65 s no prompt curto e 91,12 s contra 94,16 s no longo, diferenças dentro da variação entre repetições. O efeito da

janela maior manifesta-se em memória, não em tempo, conforme discutido na questão 3.

10) Contribuição da arquitetura local para privacidade e autonomia

A execução local elimina o envio de prompts e dados a serviços de terceiros, garantindo maior controle sobre dados sensíveis e independência de conectividade externa. Riscos residuais incluem a segurança do próprio host/container, a ausência de atualizações automáticas de segurança do modelo e a responsabilidade integral da equipe pela infraestrutura.

11) Comportamento sob múltiplos usuários simultâneos

Os dados permitem projetar o comportamento com razoável segurança. Com quatro usuários simultâneos enviando mensagens curtas, o sistema mantém a capacidade agregada — quase o dobro de requisições concluídas por unidade de tempo em relação ao atendimento sequencial — ao custo de dobrar o tempo de espera de cada um. Para uso conversacional, em que respostas curtas são a norma, esse compromisso é aceitável.

Com entradas longas, o quadro se inverte: o ganho de vazão cai para cerca de 25% enquanto a espera individual triplica, ultrapassando cinco minutos por resposta. Nesse regime o sistema deixa de ser utilizável interativamente, ainda que não apresente falhas: não houve erros, timeouts ou respostas inválidas em nenhuma das execuções.

Como a CPU já opera saturada mesmo com um único usuário, aumentar o número de slots paralelos no runtime não resolveria o problema — apenas redistribuiria o mesmo tempo de processamento entre mais requisições. Escalar esse sistema exigiria mais recursos de hardware, aceleração por GPU, ou uma camada de enfileiramento que preserve previsibilidade de espera em vez de admitir todas as requisições simultaneamente. Uma alternativa de menor custo, voltada à percepção do usuário, seria o uso de respostas em streaming, que tornaria visível o primeiro token assim que produzido em vez de manter a interface sem retorno durante toda a inferência.

12) Limitações que afetam a validade dos resultados

O truncamento fixo em 256 tokens (`num_predict`) afeta a comparabilidade da métrica tokens/s para prompts longos; a execução sob WSL2 introduz uma camada de virtualização que pode afetar a precisão de medições de tempo; o número de repetições (três) é suficiente para uma tendência, mas não elimina totalmente a variância.

13) Informações de IA generativa que precisaram de verificação

Sugestões de scripts de benchmark geradas com apoio de IA foram verificadas por execução real e conferência dos logs/JSON brutos; parâmetros como `think:false` e o uso de nonce para neutralizar o cache de prefixo foram validados empiricamente antes de serem adotados como metodologia final (ver Declaração de Uso de IA Generativa, Seção 14).

12. RELAÇÃO COM SISTEMAS OPERACIONAIS E IA GENERATIVA

O experimento evidencia, na prática, conceitos centrais da disciplina: escalonamento e concorrência (efeito da concorrência sobre latência e CPU), gerência de memória (crescimento do RSS com o tamanho do contexto, uso de `mmap` para carregar pesos), comunicação entre processos (API HTTP local entre Open WebUI e Ollama) e hierarquia de processos (relação pai-filho entre ollama e llama-server).

13. LIMITAÇÕES E AMEAÇAS À VALIDADE

- `num_predict` fixo em 256 tokens trunca respostas a prompts longos (`done_reason = length`), o que deve ser considerado ao interpretar tokens/s nesses cenários.
- A execução em WSL2 introduz uma camada de virtualização entre o Linux observado e o hardware físico, podendo afetar a precisão de métricas de tempo e CPU.
- O TTFT reportado é uma aproximação (carga do modelo + avaliação do prompt), já que as requisições foram feitas com `stream:false`, sem medição direta do primeiro token.

- Três repetições por cenário atendem ao mínimo recomendado pela atividade, mas um número maior reduziria a variância das estimativas de tokens/s e latência.
- Deriva do pico de memória ao longo da sessão. Os valores de pico de RSS crescem de forma monotônica ao longo da bateria de testes, inclusive entre repetições com parâmetros idênticos: nas três repetições da configuração de concorrência com prompt longo na máquina 2, os picos foram de 8.806 MB, 9.959 MB e 11.104 MB. Como a ordem de execução foi padrão, concorrência e contexto longo, parte da diferença de memória atribuída a cada configuração pode ser efeito da posição na sequência. Por esse motivo, a comparação de memória entre janelas de contexto apresentada na questão 3 foi restrita às execuções com processo de inferência recém-criado. Um desenho mais robusto reiniciaria o processo de inferência antes de cada execução ou alternaria a ordem das configurações.
- Pressão de memória na máquina principal. A máquina 1 dispõe de 11 GiB de RAM sob WSL2, enquanto o pico de RSS observado na configuração de contexto longo chegou a 10.266 MB, ou aproximadamente 93% da memória disponível, com swap de 4 GiB configurado. Essa proximidade do limite pode ter provocado paginação durante as execuções e afetado as medições de tempo. Um indício nesse sentido é que a máquina 1, apesar de contar com processador superior em número de núcleos, apresentou latências consistentemente maiores que a máquina 2 em praticamente todos os cenários — 108,36 s contra 94,16 s na configuração padrão com prompt longo, por exemplo. Esse resultado contraria a expectativa baseada apenas nas especificações de processador e sugere que a memória disponível, e não a capacidade de cálculo, foi o fator limitante na máquina 1.
- Ausência de medição de chamadas de sistema sob concorrência. As capturas com strace foram realizadas apenas com uma requisição em andamento, o que impede quantificar o custo de sincronização atribuível especificamente ao aumento de concorrência.

14. DECLARAÇÃO DE USO DE IA GENERATIVA

Ferramenta e modelo

Foi utilizada a interface web do Claude (Anthropic), com o modelo Claude Opus 5, durante setembro de 2026, como apoio à geração e revisão dos cinco scripts Bash de automação da atividade. A ferramenta foi utilizada majoritariamente no desenvolvimento dos scripts.

Finalidade e prompts utilizados

A ferramenta auxiliou na construção dos scripts de inventário, configuração do ambiente, monitoramento de processos, captura de chamadas de sistema e benchmark.

Prompt 1 — Geração inicial dos scripts

"Segue o enunciado da atividade em PDF. Precisamos automatizar a Parte A, a Parte B e a Parte C: inventário do ambiente, subida do Ollama no host com Open WebUI em contêiner, captura de processos e threads durante inferência ativa, captura de chamadas de sistema com strace e um benchmark comparativo. O ambiente é WSL2 sobre Windows, execução em CPU, modelo qwen3:8b. Gere scripts em Bash, um por etapa, que gravem evidências em arquivos versionáveis e produzam os dados em CSV."

Prompt 2 — Desenho experimental do benchmark

"O script de benchmark varia apenas o número de threads em três valores, o que são três pontos do mesmo eixo e não atende a seção 8 da atividade, que pede três configurações distintas. Reestruture para execução padrão, concorrência com requisições simultâneas e ajuste de janela de contexto, mantendo o mesmo modelo, com dois tamanhos de entrada e repetições por cenário."

Prompt 3 — Instrumentação de recursos

"Além dos tempos retornados pela API do Ollama, o script precisa medir consumo de CPU, memória residente, número de threads e número de processos durante cada execução. Implemente a coleta lendo o sistema de arquivos /proc, com amostragem periódica em segundo plano, gravando uma série temporal por execução e os agregados na tabela de recursos."

Prompt 4 — Captura de chamadas de sistema

"No script de strace, quando anexamos ao processo com o modelo já carregado só aparecem chamadas de sincronização e espera, sem as

operações de abertura e mapeamento dos pesos. Ajuste o script para capturar também a carga do modelo e produza tanto um resumo agregado quanto um traçado detalhado das famílias de chamadas relacionadas a arquivo, memória e rede."

Prompt 5 — Cobertura das métricas obrigatórias

"Confira se o script de benchmark cobre as métricas mínimas da seção 8.2 e implemente o que estiver faltando: tempo até a primeira resposta, critério objetivo de avaliação da resposta gerada, taxa de erro e registro de parâmetros, modelo, versão do runtime e horário de cada execução."

Validação das sugestões

As sugestões aproveitadas envolveram a organização dos scripts por etapa, o registro das evidências em arquivos versionáveis e a coleta de dados em diferentes níveis. Também foram incorporados o uso de `/proc` para monitoramento, `/proc/uptime` como relógio monotônico, a identificação dos processos do Ollama por descendência, o descarregamento do modelo antes do `strace` e o uso de marcadores únicos para evitar interferências do cache de prompt.

Durante os testes, foram identificados quatro problemas de instrumentação: uso de relógio de parede no WSL2, exclusão do processo runner do monitoramento, aquecimento com prompt diferente do experimento e truncamento dos argumentos capturados pelo `strace`. Todos foram corrigidos antes da execução definitiva.

As sugestões também foram criticamente avaliadas. O aquecimento com o próprio prompt do experimento foi rejeitado por eliminar o custo de leitura do prompt, que fazia parte do fenômeno analisado. Em outro caso, uma explicação sobre valores nulos na coleta de métricas foi descartada após teste direto demonstrar que a causa estava no conjunto de processos monitorado.

Todos os scripts passaram por `bash -n` e por execução completa em ambiente de teste. Os campos utilizados de `/proc/<pid>/stat` foram conferidos na documentação de `proc(5)`. A medição de tempo foi validada pela comparação com `total_duration` do servidor, apresentando divergência máxima de 107 ms nas 36

requisições. A identificação dos processos foi confirmada pelos valores de pico de processos e memória, compatíveis com a execução do modelo.

15. CONCLUSÃO

A execução local do Qwen3-8B via Ollama, testada em duas máquinas independentes e em três configurações — padrão, concorrência e contexto longo —, permitiu observar de forma concreta como decisões de configuração afetam processos, threads, uso de processador e memória, e a responsividade do sistema.

O resultado central é que a concorrência não melhora nem piora o desempenho de forma absoluta: ela troca responsividade individual por capacidade agregada. Com quatro requisições simultâneas de prompt curto, cada usuário esperou aproximadamente o dobro do tempo, mas o sistema concluiu quase o dobro de requisições por unidade de tempo, consumindo cerca de 45% menos processamento por requisição — um ganho de eficiência por amortização de trabalho entre requisições agrupadas. Esse compromisso, contudo, deteriora-se com entradas longas, em que o ganho de vazão cai para cerca de um quarto enquanto a espera individual triplica.

O tamanho da entrada mostrou-se o fator de maior impacto sobre a experiência de uso: um prompt de 1.794 tokens elevou o tempo até a primeira resposta de 1,40 s para 54,31 s, um fator de 39, concentrado integralmente na fase de avaliação do prompt. Já o tamanho da janela de contexto afetou memória sem afetar tempo, reservando cerca de 2,3 GB adicionais ao ser dobrada, independentemente do tamanho do prompt efetivamente enviado.

Esses resultados reforçam, na prática, os conceitos de escalonamento, concorrência e gerência de memória estudados na disciplina, e evidenciam que a avaliação de um sistema local de IA generativa exige distinguir métricas orientadas ao usuário, como latência e tempo até a primeira resposta, de métricas orientadas ao sistema, como vazão e custo de processamento por requisição — métricas que, como se verificou, podem apontar em direções opostas diante da mesma decisão de configuração.

16. REFERÊNCIAS

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Operating System Concepts. 10. ed. Hoboken: Wiley, 2018.

TANENBAUM, A. S.; BOS, H. Modern Operating Systems. 5. ed. Boston: Pearson, 2023.

KERRISK, M. The Linux Programming Interface. San Francisco: No Starch Press, 2010.

LINUX MAN-PAGES PROJECT. fork(2), execve(2), wait(2), clone(2), pthreads(7), sched(7). Disponível em: <https://man7.org/linux/man-pages/>.

OLLAMA. Repositório oficial. Disponível em: <https://github.com/ollama/ollama>.

OPEN WEBUI. Repositório oficial. Disponível em: <https://github.com/open-webui/open-webui>.

HUGGING FACE. Qwen/Qwen3-8B. Disponível em: <https://huggingface.co/Qwen/Qwen3-8B>.